
DSC5001: Statistical Machine Learning I

Week 5: Regression II: Nonlinear Regression and Splines

Kangning Cui

City University of Hong Kong (Dongguan)

Semester A, 2026/27

Learning objectives

1. Represent curved relationships using transformed features.
2. Explain what happens when a model is too simple or too flexible.
3. Construct polynomial and cubic regression-spline models.
4. Use validation to select polynomial degree and spline complexity.

Nonlinear mean functions

DSC5001 ■ Semester A, 2026/27

From Linear to Nonlinear Mean Functions

In Week 4, linear regression modeled the conditional mean as

$$\mathbb{E}[Y \mid X = x] = \beta_0 + \beta_1 x.$$

The slope is constant, so the fitted mean can only be a straight line.
More generally, regression models

$$\mathbb{E}[Y \mid X = x] = f(x),$$

or equivalently

$$Y = f(X) + \varepsilon, \quad \mathbb{E}[\varepsilon \mid X] = 0.$$

Main idea

The regression target is still the conditional mean; only the family used to represent it becomes richer.

The Straight-Line Restriction

For

$$f(x) = \beta_0 + \beta_1 x,$$

the derivative is

$$f'(x) = \beta_1.$$

Therefore the fitted rate of change is the same everywhere.

For any change Δx ,

$$f(x + \Delta x) - f(x) = \beta_1 \Delta x.$$

A straight-line model cannot represent a relationship whose slope changes across the input range.

A Curved Mean Function

Suppose the conditional mean follows the pattern

x	-2	-1	0	1	2
$\mathbb{E}[Y \mid X = x]$	5	2	1	2	5

The mean decreases and then increases.

A straight line cannot reproduce both behaviors.

A simple curved model is

$$f(x) = 1 + x^2.$$

Main idea

Curvature may belong to the population mean itself; it is not necessarily caused by random noise.

Polynomial Regression

A degree- d polynomial regression model has the form

$$f_d(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_d x^d.$$

Examples:

$$d = 1 : \quad \beta_0 + \beta_1 x,$$

$$d = 2 : \quad \beta_0 + \beta_1 x + \beta_2 x^2,$$

$$d = 3 : \quad \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3.$$

Increasing d gives the model more freedom to represent curvature.

Nonlinear in the Input, Linear in the Coefficients

Consider the quadratic model

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2.$$

It is nonlinear as a function of x because of the term x^2 .

However, it is linear in the unknown coefficients:

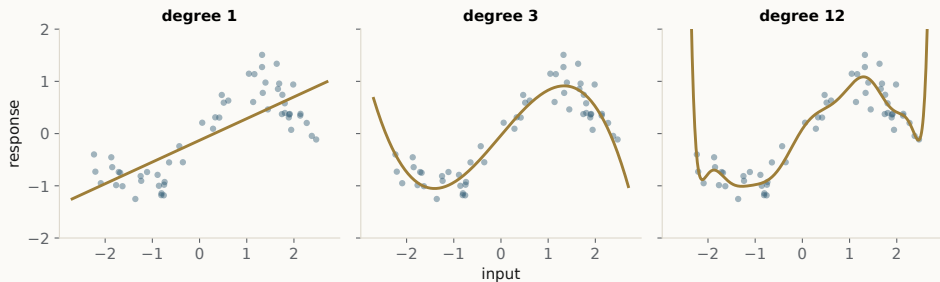
$$\beta_0, \quad \beta_1, \quad \beta_2.$$

There are no terms such as

$$\beta_1^2, \quad \beta_1 \beta_2, \quad e^{\beta_2}.$$

Polynomial regression produces a nonlinear curve but can still be fitted using linear least squares.

Polynomial Degree and Fitted Shape



Degree 1
rigid global shape

Degree 3
moderate curvature

Degree 12
high global flexibility

Takeaway. Changing polynomial degree changes the set of curves the model can represent.

Transformed Features

A quadratic model can be written using the feature map

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \end{bmatrix}.$$

Let

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}.$$

Then

$$f(x) = \phi(x)^\top \beta.$$

The nonlinear model is therefore a linear combination of transformed features.

Feature Construction

Suppose

$$x = -2, -1, 0, 1, 2.$$

For a quadratic model, each scalar input is transformed into

$$\phi(x) = [1, x, x^2]^\top.$$

For example,

$$\phi(-2) = [1, -2, 4]^\top, \quad \phi(1) = [1, 1, 1]^\top.$$

Main idea

The extra predictors are constructed from the original input; they are not additional measurements.

Polynomial Design Matrix

For observations $x_1, \dots, x_n$, a degree- d polynomial produces

$$\Phi = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^d \\ 1 & x_2 & x_2^2 & \cdots & x_2^d \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^d \end{bmatrix}.$$

The regression model becomes

$$\mathbf{y} = \Phi\boldsymbol{\beta} + \boldsymbol{\varepsilon}.$$

This has the same form as the multiple linear regression model from Week 4.

Polynomial Regression Uses OLS

The least-squares problem is

$$\hat{\beta} = \arg \min_{\beta} \|\mathbf{y} - \Phi\beta\|_2^2.$$

If Φ has full column rank,

$$\hat{\beta} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{y}.$$

The OLS machinery is unchanged.

What changes is the representation:

$$x \longrightarrow [1, x, x^2, \dots, x^d]^\top.$$

Prediction with Polynomial Features

After fitting the model, a new input must be transformed in exactly the same way. For a quadratic model and new input x_0 ,

$$\phi(x_0) = [1, x_0, x_0^2]^\top.$$

The predicted conditional mean is

$$\hat{f}(x_0) = \phi(x_0)^\top \hat{\beta}.$$

For $x_0 = 3$, the model uses

$$\phi(3) = [1, 3, 9]^\top.$$

Common mistake

The feature transformation used at prediction time must match the transformation used during fitting.

Degree Controls Model Flexibility

A degree-2 polynomial is contained inside the degree-3 model:

$$\beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3.$$

Setting $\beta_3 = 0$ recovers a quadratic model.

Define

$$\mathcal{F}_d = \{ \beta_0 + \beta_1 x + \cdots + \beta_d x^d : \beta_0, \dots, \beta_d \in \mathbb{R} \},$$

the set of all polynomial models with degree at most d .

Therefore,

$$\mathcal{F}_1 \subset \mathcal{F}_2 \subset \cdots \subset \mathcal{F}_d.$$

Higher degree means a larger set of candidate functions.

Training Error and Polynomial Degree

Because the model spaces are nested,

$$\mathcal{F}_d \subset \mathcal{F}_{d+1},$$

the larger model can always reproduce the smaller one.

Therefore, on the same training data,

$$RSS_{d+1} \leq RSS_d.$$

Similarly,

$$MSE_{\text{train},d+1} \leq MSE_{\text{train},d}.$$

Training error alone will systematically favor more flexible polynomial models.

High-Degree Polynomial Features

High-degree polynomial models can create practical difficulties.

If $x = 10$,

$$x = 10, \quad x^2 = 100, \quad x^5 = 100000.$$

Also, the feature columns

$$x, \quad x^2, \quad x^3, \dots$$

can be strongly related.

Possible consequences include:

- ▶ unstable coefficient estimates;
- ▶ poor numerical conditioning;
- ▶ strong boundary behavior.

These problems become more important as degree increases.

Underfitting and Overfitting

Underfitting

The model is too restrictive to represent important systematic structure.

Overfitting

The fitted model adapts too strongly to sample-specific variation and predicts new data poorly.

A model can therefore fail in two different ways:

too little flexibility or too much flexibility.

Training error alone cannot distinguish these cases.

Training Fit Is Not Enough

Suppose four polynomial models produce

Degree	1	3	6	12
Training MSE	5.8	3.1	1.7	0.4

Based only on training error, degree 12 appears best.

But the real modeling question is:

Which model predicts new observations best?

To answer that question, we need prediction error rather than training fit.

Prediction error

DSC5001 ■ Semester A, 2026/27

Training Error

Training error measures how well the fitted model predicts the observations used to estimate it. For squared-error regression,

$$\hat{R}_{\text{train}} = \frac{1}{n} \sum_{i=1}^n \left(y_i - \hat{f}(x_i) \right)^2.$$

The same observations are used for both:

fitting the model and measuring the error.

Main idea

Training error describes fit to the observed sample, not necessarily prediction on future data.

Test Error

Test error concerns observations that were not used to estimate the model.

Let (X_0, Y_0) be a new observation from the target population.

The expected prediction error is

$$R_{\text{test}} = \mathbb{E} \left[\left(Y_0 - \hat{f}(X_0) \right)^2 \right].$$

Training error asks:

How well did we fit the observed sample?

Test error asks:

How well will we predict new data?

Training and Test Error Are Different

Consider three candidate polynomial models.

Degree	1	4	12
Training MSE	5.2	2.1	0.6
Test MSE	5.5	2.8	4.3

The degree-12 model gives the smallest training error.
But it does not give the smallest test error.

The model that fits the training data best need not predict new data best.

Why Can the Two Errors Behave Differently?

Increasing model flexibility gives the fitted model more freedom. On the training sample, this freedom can be used to reduce residuals. Therefore training error usually decreases as flexibility increases. But the fitted model may also become more sensitive to:

- ▶ random noise;
- ▶ unusual observations;
- ▶ which particular sample was collected.

This sample sensitivity can increase prediction error on new data.

Prediction Error at a Fixed Input

To understand prediction error more carefully, fix an input value x_0 .
Assume a future response satisfies

$$Y_0 = f(x_0) + \varepsilon_0,$$

where

$$\mathbb{E}[\varepsilon_0] = 0, \quad \text{Var}(\varepsilon_0) = \sigma^2.$$

The fitted model predicts

$$\hat{f}_D(x_0),$$

where D denotes the training dataset used to fit the model.

We want to understand

$$\mathbb{E}_{D, \varepsilon_0} \left[\left(Y_0 - \hat{f}_D(x_0) \right)^2 \right].$$

What Is Random Here?

At the fixed input x_0 , there are two sources of randomness.

1. Future observation noise

$$\varepsilon_0.$$

Even if the true mean $f(x_0)$ were known exactly, the future response would still vary.

2. Training-sample randomness

$$D.$$

Different training samples generally produce different fitted functions:

$$\hat{f}_D(x_0).$$

Main idea

Prediction error reflects both uncertainty in the future outcome and uncertainty in the fitted model.

The Average Fitted Prediction

Imagine repeatedly drawing new training datasets of the same size.
Each dataset produces a fitted prediction:

$$\hat{f}_{D_1}(x_0), \hat{f}_{D_2}(x_0), \hat{f}_{D_3}(x_0), \dots$$

Define their population average as

$$m(x_0) = \mathbb{E}_D \left[\hat{f}_D(x_0) \right].$$

This is the prediction produced by the learning procedure on average over repeated training samples.

Bias

The true conditional mean at x_0 is

$$f(x_0).$$

The average prediction of the learning procedure is

$$m(x_0) = \mathbb{E}_D[\hat{f}_D(x_0)].$$

Define the bias as

$$\text{Bias}[\hat{f}_D(x_0)] = m(x_0) - f(x_0).$$

Therefore,

$$\text{Bias}[\hat{f}_D(x_0)]^2 = [m(x_0) - f(x_0)]^2.$$

Main idea

Bias measures systematic error across repeated training samples, not the error of one fitted model on one observation.

Variance

Different training datasets produce different fitted predictions.

Their variability is

$$\text{Var}_D[\hat{f}_D(x_0)] = \mathbb{E}_D \left[\left(\hat{f}_D(x_0) - m(x_0) \right)^2 \right].$$

A model has high variance at x_0 if its fitted prediction changes substantially when the training sample changes.

Main idea

Variance measures sensitivity of the learning procedure to the particular sample used for fitting.

Bias–Variance Decomposition: Step 1

Start from the prediction error

$$Y_0 - \hat{f}_D(x_0).$$

Using

$$Y_0 = f(x_0) + \varepsilon_0,$$

we obtain

$$Y_0 - \hat{f}_D(x_0) = f(x_0) + \varepsilon_0 - \hat{f}_D(x_0).$$

Now add and subtract $m(x_0)$:

$$Y_0 - \hat{f}_D(x_0) = \varepsilon_0 + [f(x_0) - m(x_0)] + [m(x_0) - \hat{f}_D(x_0)].$$

This separates three different sources of prediction error.

Bias–Variance Decomposition: Step 2

For shorter notation, write

$$f_0 = f(x_0), \quad m = m(x_0), \quad \hat{f} = \hat{f}_D(x_0).$$

Then

$$Y_0 - \hat{f} = \varepsilon_0 + (f_0 - m) + (m - \hat{f}).$$

Squaring gives

$$\begin{aligned}(Y_0 - \hat{f})^2 &= \varepsilon_0^2 + (f_0 - m)^2 + (m - \hat{f})^2 \\ &\quad + 2\varepsilon_0(f_0 - m) + 2\varepsilon_0(m - \hat{f}) \\ &\quad + 2(f_0 - m)(m - \hat{f}).\end{aligned}$$

The next step is to take expectation.

Why the First Cross Terms Vanish

Because

$$\mathbb{E}[\varepsilon_0] = 0,$$

the first cross term satisfies

$$\mathbb{E}[2\varepsilon_0(f_0 - m)] = 2(f_0 - m)\mathbb{E}[\varepsilon_0] = 0.$$

Also, future noise is independent of the training dataset D .

Therefore,

$$\mathbb{E}[\varepsilon_0(m - \hat{f})] = \mathbb{E}[\varepsilon_0] \mathbb{E}_D[m - \hat{f}] = 0.$$

Why the Final Cross Term Vanishes

Recall that

$$m = \mathbb{E}_D[\hat{f}].$$

Therefore,

$$\mathbb{E}_D[m - \hat{f}] = m - \mathbb{E}_D[\hat{f}] = 0.$$

Hence

$$\mathbb{E} \left[2(f_0 - m)(m - \hat{f}) \right] = 0.$$

All three cross terms disappear after taking expectation.

Bias–Variance Decomposition

The remaining three terms are

$$\mathbb{E}[\varepsilon_0^2] = \sigma^2,$$

$$(f(x_0) - m(x_0))^2 = \text{Bias}[\hat{f}_D(x_0)]^2,$$

and

$$\mathbb{E}_D \left[\left(\hat{f}_D(x_0) - m(x_0) \right)^2 \right] = \text{Var}_D[\hat{f}_D(x_0)].$$

Therefore,

$$\mathbb{E} \left[\left(Y_0 - \hat{f}_D(x_0) \right)^2 \right] = \sigma^2 + \text{Bias}[\hat{f}_D(x_0)]^2 + \text{Var}_D[\hat{f}_D(x_0)]$$

Three Components of Prediction Error

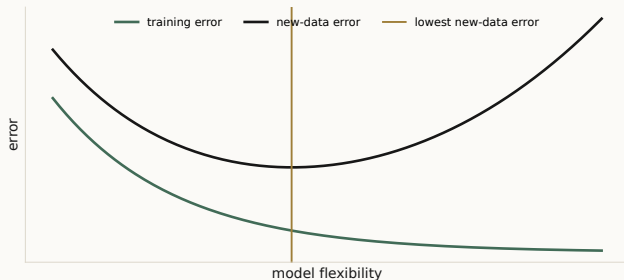
Component	Meaning	Typical source
Noise	variation in future outcomes around the true mean	data-generating process
Squared bias	systematic difference between the average fitted model and the true mean	model too restrictive
Variance	variation of the fitted prediction across training samples	sensitivity to training data

The noise term

$$\sigma^2$$

is irreducible unless additional predictive information changes the data-generating uncertainty.

Training and Test Error



Green curve
training error

Black curve
new-data error

Gold marker
best tested flexibility

Takeaway. Training error can keep falling after new-data error has started to increase.

How Flexibility Changes Bias and Variance

A very simple model usually has limited ability to represent the true mean function.
This tends to produce:

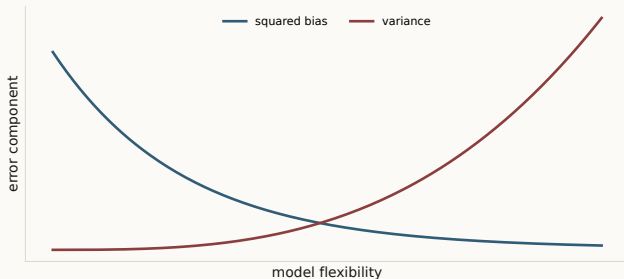
large squared bias and small variance.

A very flexible model can adapt closely to the training observations.
This often produces:

smaller squared bias but larger variance.

The prediction error depends on the balance between these components.

Squared Bias and Variance



Blue curve
squared bias

Red curve
variance

Horizontal axis
model flexibility

Takeaway. Increasing flexibility often reduces bias while increasing sensitivity to the training sample.

Bias and Variance Are Repeated-Sample Concepts

Suppose one fitted degree-12 polynomial looks highly irregular.
That observation alone does not directly measure its variance.
Variance refers to what happens across repeated datasets:

$$D_1, D_2, \dots$$

If the fitted curve changes greatly from one dataset to another, then

$$\text{Var}_D[\hat{f}_D(x)]$$

is large.

Likewise, bias compares the true function with the average fitted function across repeated samples.

Main idea

Bias and variance describe a learning procedure, not just one realized fitted curve.

Underfitting and Overfitting Revisited

The bias–variance view helps explain the two extremes.

	Underfitting	Overfitting
Flexibility	too low	too high
Training error	often high	often very low
Squared bias	often high	often lower
Variance	often low	often high
New-data error	can be high	can be high

The goal is not maximum flexibility.

The goal is useful predictive flexibility.

Read the Decomposition

Check your understanding. A very flexible model changes substantially when a small number of training observations are replaced. Which component of the bias-variance decomposition is likely to be large?

Use the decomposition

$$\text{prediction error} = \text{noise} + \text{bias}^2 + \text{variance}$$

to justify your answer.

What the Decomposition Does Not Tell Us

The decomposition explains why prediction error can change with model flexibility.
But it does not directly tell us:

which polynomial degree should we choose?

The true test error

$$R_{\text{test}}$$

is unknown when we fit the model.

We therefore need a practical way to estimate predictive performance without using the final test data for model selection.

Later: validation and cross-validation.

Piecewise polynomial models

DSC5001 ■ Semester A, 2026/27

The Limitation of a Global Polynomial

A degree- d polynomial uses one formula over the entire input range:

$$f(x) = \beta_0 + \beta_1 x + \cdots + \beta_d x^d.$$

Changing one coefficient changes the fitted function everywhere.

If additional flexibility is needed only in one part of the input range, a global polynomial may change unnecessarily in other regions.

Main idea

Global polynomial flexibility is difficult to localize.

Why Not Just Increase the Degree?

A higher polynomial degree creates more possible curve shapes. However, increasing the degree affects the complete function:

$$f(x) = \sum_{j=0}^d \beta_j x^j.$$

Possible consequences include:

- ▶ strong oscillation in some regions;
- ▶ unstable behavior near boundaries;
- ▶ poor extrapolation;
- ▶ flexibility added where it may not be needed.

The alternative is to introduce flexibility locally.

Piecewise Polynomial Models

Instead of using one polynomial over the entire range, divide the input space into regions. For example, with one dividing point ξ ,

$$f(x) = \begin{cases} p_1(x), & x \leq \xi, \\ p_2(x), & x > \xi, \end{cases}$$

where p_1 and p_2 are polynomial functions.

The two regions can now have different polynomial behavior.

Main idea

Piecewise models allow the curve to adapt differently in different parts of the input space.

Knots

The locations where neighboring polynomial pieces meet are called **knots**.
With K interior knots,

$$\xi_1 < \xi_2 < \cdots < \xi_K,$$

the input range is divided into $K + 1$ regions.

For example, three knots produce

$$(-\infty, \xi_1], \quad (\xi_1, \xi_2], \quad (\xi_2, \xi_3], \quad (\xi_3, \infty).$$

A knot is a model-design location; it does not have to coincide with an observed input value.

A Piecewise Cubic with One Knot

Suppose there is one knot at ξ .

A completely unrestricted piecewise cubic can be written as

$$f(x) = \begin{cases} a_0 + a_1x + a_2x^2 + a_3x^3, & x \leq \xi, \\ b_0 + b_1x + b_2x^2 + b_3x^3, & x > \xi. \end{cases}$$

There are eight coefficients:

$$a_0, \dots, a_3, \quad b_0, \dots, b_3.$$

Without additional restrictions, the two cubic pieces do not have to join smoothly.

What Can Go Wrong at a Knot?

Two independent polynomial pieces may meet badly.

At $x = \xi$, we could have

$$f(\xi^-) \neq f(\xi^+),$$

which creates a jump.

Even if the values match, we could still have

$$f'(\xi^-) \neq f'(\xi^+),$$

which creates a sharp corner.

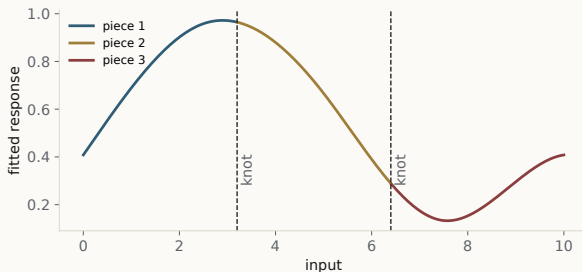
We therefore need constraints controlling how neighboring pieces meet.

Piecewise Curves Join at Knots

- ▶ Colored segments represent local polynomial pieces.
- ▶ Dashed vertical lines indicate knot locations.
- ▶ Smoothness constraints connect the pieces into one curve.

Main idea

A knot allows local behavior to change without forcing the fitted curve to break.



Continuity of the Fitted Function

The first requirement is that the curve itself does not jump at a knot.
For knot ξ ,

$$f(\xi^-) = f(\xi^+).$$

Here,

$$f(\xi^-)$$

denotes the value approached from the left, while

$$f(\xi^+)$$

denotes the value approached from the right.

This condition makes the fitted curve continuous.

Continuity of the First Derivative

Matching the function values removes jumps, but the curve could still have a sharp corner.
To remove the corner, require

$$f'(\xi^-) = f'(\xi^+).$$

The left and right polynomial pieces then have the same slope at the knot.

Main idea

Matching the first derivative makes the direction of the curve change smoothly across the knot.

Continuity of the Second Derivative

A cubic spline also requires the curvature to match:

$$f''(\xi^-) = f''(\xi^+).$$

Therefore, at every interior knot ξ_k ,

$$\begin{aligned} f(\xi_k^-) &= f(\xi_k^+), \\ f'(\xi_k^-) &= f'(\xi_k^+), \\ f''(\xi_k^-) &= f''(\xi_k^+). \end{aligned}$$

These three constraints produce a smooth transition between neighboring cubic pieces.

Cubic Splines

A **cubic spline** is a function that:

1. is cubic between neighboring knots;
2. is continuous at every interior knot;
3. has a continuous first derivative;
4. has a continuous second derivative.

Thus,

$$f, \quad f', \quad f''$$

are continuous across each knot.

Common mistake

Continuity of the third derivative is not required for a cubic spline.

What Kind of Basis Function Do We Need?

At a knot ξ , we want the fitted curve to behave differently on the two sides.
But we also require

$$f, \quad f', \quad f''$$

to remain continuous at ξ .

So we need a new feature that:

- ▶ has no effect before the knot;
- ▶ changes the cubic behavior after the knot;
- ▶ does not create a jump, corner, or curvature discontinuity at the knot.

This motivates the truncated cubic feature

$$(x - \xi)_+^3.$$

The Positive-Part Function

Define the positive part of a number u by

$$u_+ = \max(u, 0).$$

Therefore,

$$(x - \xi)_+ = \begin{cases} 0, & x \leq \xi, \\ x - \xi, & x > \xi. \end{cases}$$

It acts like a switch:

$x \leq \xi \Rightarrow \text{inactive}, \quad x > \xi \Rightarrow \text{active}.$
--

The positive-part operation tells us *where* a new basis function begins to affect the curve.

Why Do We Raise It to the Third Power?

We are constructing a **cubic** spline.

So after the knot, we want the new feature to contribute a cubic term:

$$(x - \xi)^3.$$

Before the knot, we want no contribution at all.

Combining these requirements gives

$$h_{\xi}(x) = (x - \xi)_+^3 = \begin{cases} 0, & x \leq \xi, \\ (x - \xi)^3, & x > \xi. \end{cases}$$

The power 3 is not arbitrary:

cubic spline $\implies$ truncated cubic term.

What Does the Truncated Cubic Do?

Start with a global cubic polynomial

$$p(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3.$$

Now add one truncated cubic term:

$$f(x) = p(x) + \gamma(x - \xi)_+^3.$$

Then

$$f(x) = \begin{cases} p(x), & x \leq \xi, \\ p(x) + \gamma(x - \xi)^3, & x > \xi. \end{cases}$$

Before the knot, nothing changes.

After the knot, the additional cubic term changes the local shape.

A Numerical Example

Let the knot be

$$\xi = 2.$$

Consider

$$h(x) = (x - 2)_+^3.$$

Then

$$h(1) = 0, \quad h(2) = 0, \quad h(3) = 1, \quad h(4) = 8.$$

So the feature contributes nothing for

$$x \leq 2,$$

but becomes increasingly active after the knot.

Main idea

The coefficient multiplying this feature controls how the cubic behavior changes after $x = 2$.

Why Does It Join Smoothly?

Let

$$h_{\xi}(x) = (x - \xi)_+^3.$$

Its first derivative is

$$h'_{\xi}(x) = \begin{cases} 0, & x \leq \xi, \\ 3(x - \xi)^2, & x > \xi, \end{cases}$$

and its second derivative is

$$h''_{\xi}(x) = \begin{cases} 0, & x \leq \xi, \\ 6(x - \xi), & x > \xi. \end{cases}$$

At the knot,

$$h_{\xi}(\xi) = 0, \quad h'_{\xi}(\xi) = 0, \quad h''_{\xi}(\xi) = 0.$$

So adding this feature does not break continuity of

$$f, \quad f', \quad f''.$$

The Third Derivative Is Allowed to Change

For

$$h_{\xi}(x) = (x - \xi)_+^3,$$

the third derivative is

$$h_{\xi}'''(x) = \begin{cases} 0, & x < \xi, \\ 6, & x > \xi. \end{cases}$$

So the third derivative changes at the knot.

This is exactly what allows the cubic behavior on the right side to differ from that on the left.

A cubic spline requires continuity of

$$f, \quad f', \quad f'',$$

but not of

$$f'''.$$

One Knot Gives One Extra Cubic Direction

With one knot ξ ,

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \gamma(x - \xi)_+^3.$$

The corresponding basis is

$$1, \quad x, \quad x^2, \quad x^3, \quad (x - \xi)_+^3.$$

The first four basis functions describe a global cubic.

The fifth basis function allows the cubic behavior to change after the knot.

Therefore a one-knot cubic spline has

5

free coefficients.

Why Are There Five Free Coefficients?

With one knot, there are two cubic regions.

Without any constraints:

$$4 + 4 = 8$$

coefficients are needed.

At the knot, cubic-spline smoothness imposes three conditions:

$$f(\xi^-) = f(\xi^+),$$

$$f'(\xi^-) = f'(\xi^+),$$

$$f''(\xi^-) = f''(\xi^+).$$

Therefore,

$$8 - 3 = 5.$$

This matches the five basis functions in

$$1, x, x^2, x^3, (x - \xi)_+^3.$$

Several Knots

Suppose there are K interior knots:

$$\xi_1 < \xi_2 < \cdots < \xi_K.$$

Add one truncated cubic feature for each knot:

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_{k=1}^K \gamma_k (x - \xi_k)_+^3.$$

Each term

$$(x - \xi_k)_+^3$$

allows the cubic behavior to change after knot ξ_k .

The function remains cubic between neighboring knots.

Cubic Regression-Spline Basis

With K interior knots, define

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \\ (x - \xi_1)_+^3 \\ \vdots \\ (x - \xi_K)_+^3 \end{bmatrix}.$$

Then

$$f(x) = \phi(x)^\top \theta.$$

The number of basis functions is

$$K + 4.$$

A cubic regression spline is nonlinear in x but linear in its coefficient vector.

Why $K + 4$ Parameters?

Each cubic polynomial has four coefficients:

$$a + bx + cx^2 + dx^3.$$

With K knots, the input range is divided into $K + 1$ regions.
If each region had its own independent cubic, we would need

$$4(K + 1)$$

coefficients.

At each knot, the two neighboring cubics must satisfy

$$\begin{aligned} f_L(\xi_k) &= f_R(\xi_k), \\ f'_L(\xi_k) &= f'_R(\xi_k), \quad f''_L(\xi_k) = f''_R(\xi_k). \end{aligned}$$

So each knot imposes three constraints. Therefore the number of free coefficients is

$$4(K + 1) - 3K = K + 4.$$

Cubic Spline and Spline Basis

A cubic spline is a function that is:

- ▶ cubic between neighboring knots;
- ▶ continuous at each knot;
- ▶ continuous in its first and second derivatives.

The same cubic spline can be represented using different sets of basis functions.

For example:

truncated-power basis

or

B-spline basis.

The spline model stays the same; only its mathematical representation changes.

Why Use Another Basis?

The truncated-power representation is easy to understand.

But consider one basis function:

$$(x - \xi_1)_+^3.$$

Once $x > \xi_1$, it remains active for every larger value of x .

So changing its coefficient can affect a large part of the fitted curve.

Similarly, several truncated-power basis functions can be active at the same time over a large region.

This can make the design matrix numerically less stable.

Main idea

We would prefer basis functions whose effects are more local.

What Does Local Support Mean?

A function has **local support** if it is nonzero only over a limited interval.
For example, a basis function $B_j(x)$ may satisfy

$$B_j(x) = 0$$

outside one small region of the input range.

So changing its coefficient mainly changes the fitted curve in that region.

By contrast, a feature such as

$$x^3$$

is active across essentially the whole input range.

B-Spline Basis Functions

A B-spline basis is a collection of local basis functions

$$B_1(x), B_2(x), \dots, B_M(x).$$

The fitted cubic spline is written as

$$f(x) = \sum_{j=1}^M \theta_j B_j(x).$$

Here:

M = number of B-spline basis functions,

and

θ_j = coefficient multiplying basis function $B_j(x)$.

For the cubic spline space considered here, with K interior knots,

$$M = K + 4.$$

What Changes with a B-Spline Basis?

Using the truncated-power basis:

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_{k=1}^K \gamma_k (x - \xi_k)_+^3.$$

Using a B-spline basis:

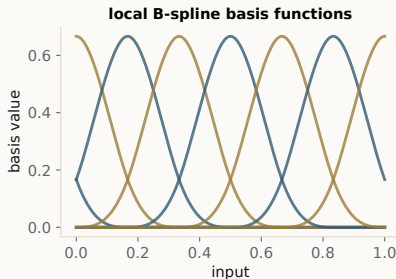
$$f(x) = \sum_{j=1}^M \theta_j B_j(x).$$

These are two different ways to represent a cubic spline.

The important difference is:

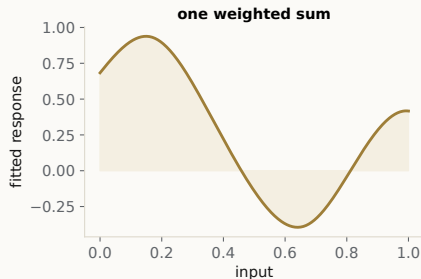
Truncated-power basis	B-spline basis
basis functions can remain active globally less numerically convenient	basis functions are locally supported usually better conditioned

B-Spline Basis Functions Cover the Input Range



Each curve
one basis function

Local support
active only locally



Final spline
weighted sum

Takeaway. Several local basis functions combine to produce one smooth cubic spline.

How the B-Spline Fit Is Built

At one input x , several nearby B-spline basis functions may be nonzero.
The prediction is their weighted sum:

$$f(x) = \theta_1 B_1(x) + \cdots + \theta_M B_M(x).$$

If

$$B_j(x) = 0,$$

then coefficient θ_j contributes nothing at that input.

Therefore each coefficient mainly controls a local part of the fitted curve.

Fitting and selecting nonlinear models

DSC5001 ■ Semester A, 2026/27

Validation Error

Let c denote a model-complexity choice, such as:

- ▶ polynomial degree;
- ▶ number of spline knots.

Fit each candidate model on the training data and measure its error on separate validation data:

$$\hat{R}_{\text{val}}(c) = \frac{1}{n_{\text{val}}} \sum_{i \in \text{val}} \left(y_i - \hat{f}_c(x_i) \right)^2.$$

Choose

$$\hat{c} = \arg \min_c \hat{R}_{\text{val}}(c).$$

Main idea

Training data estimate the coefficients; validation data choose the model complexity.

Cross-Validation

When a separate validation set is too costly, divide the available training data into K folds:

$$I_1, \dots, I_K.$$

For fold k , let

$$\hat{f}_c^{(-k)}$$

denote the model fitted without observations in I_k .

The K -fold cross-validation error is

$$\text{CV}_K(c) = \frac{1}{n} \sum_{k=1}^K \sum_{i \in I_k} \left(y_i - \hat{f}_c^{(-k)}(x_i) \right)^2.$$

Every observation is used for validation once and for training in the other folds.

From Model Selection to Final Evaluation

Validation or cross-validation is used to choose

$\hat{c}$.

A typical workflow is

fit candidates $\longrightarrow$ compare validation error $\longrightarrow$ choose $\hat{c}$ $\longrightarrow$ final evaluation.

After selecting the model specification, it can be refitted using all available non-test data. The test set is then used once for final evaluation.

Common mistake

If the test set is repeatedly used to choose models, it is no longer an independent test set.

Fitting a Regression Spline

Suppose the spline is represented by basis functions

$$B_1(x), \dots, B_M(x),$$

where M is the number of basis functions.

For the cubic spline space used here, with K interior knots,

$$M = K + 4.$$

Construct the design matrix using

$$B_{ij} = B_j(x_i), \quad \mathbf{B} \in \mathbb{R}^{n \times M}.$$

Then estimate the coefficients by least squares:

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \|\mathbf{y} - \mathbf{B}\boldsymbol{\theta}\|_2^2.$$

The fitted spline is

$$\hat{f}(x) = \sum_{j=1}^M \hat{\theta}_j B_j(x).$$

Knot Number and Placement

The number of knots controls how much local flexibility the spline has.
For a cubic spline with K interior knots,

$$M = K + 4.$$

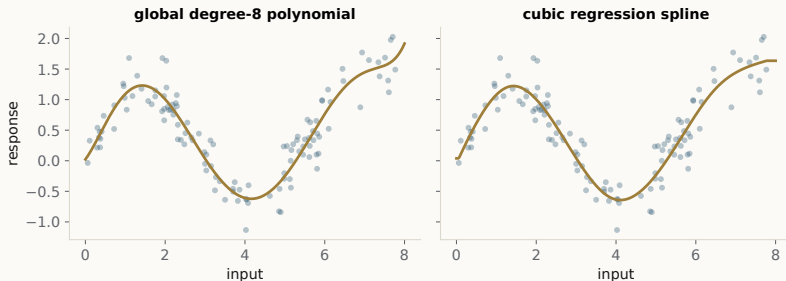
More knots give more basis functions and a larger spline space.
Knot locations also matter:

- ▶ equally spaced knots divide the input range geometrically;
- ▶ quantile-based knots place more knots where observations are dense.

Main idea

Cross-validation can compare candidate knot choices using prediction error rather than training fit alone.

Polynomials versus Splines



Data

same observations

Polynomial

global flexibility

Spline

more local flexibility

Takeaway. Polynomials and splines differ mainly in how flexibility is distributed across the input range.

Additive Models with Splines

For several predictors

$$\mathbf{x} = (x_1, \dots, x_p),$$

an additive model can assign a separate smooth function to each predictor:

$$\mathbb{E}[Y \mid \mathbf{x}] = \beta_0 + f_1(x_1) + \dots + f_p(x_p).$$

Each function f_j can be represented using a spline basis.

This allows each predictor to have a nonlinear but interpretable smooth effect.

Common mistake

An additive model does not include interactions unless interaction terms are added explicitly.

Summary

- ▶ Polynomial regression represents curvature through transformed global features.
- ▶ Splines use knots to introduce more local flexibility.
- ▶ B-splines provide a local basis for representing cubic splines.
- ▶ Least squares estimates the basis coefficients.
- ▶ Validation or cross-validation selects model complexity.

Week 5: connect the statistical question, mathematics, and evidence

Questions and discussion